

HMSpectralGun User Manual

Turbospectrum-based stellar spectral synthesis and line-by-line abundance fitting

Improved LaTeX edition

Project: HMSpectralGun

Scope: User-facing execution, input preparation, and validation workflow

Status: Operational manual

Language: English

May 27, 2026

Contents

1 Purpose and Scope	2
2 Recommended Quick Start	2
3 Requirements	2
3.1 Python environment	2
3.2 Turbospectrum installation	3
4 Input File Structure	3
4.1 The <code>input.ts</code> file	3
4.2 Spectrum table columns	4
4.3 Abundance file: <code>abu.ts</code>	4
4.4 Line-list files	4
5 Workflow 1: Classical Serial Synthesis	5
5.1 Command	5
5.2 Execution logic	5
6 Workflow 2: Parallel Synthesis	5
6.1 Command	5
6.2 Execution logic	5
7 Workflow 3: Line-by-line Chi-square Analysis	6
7.1 Configuration file	6
7.2 Command	6
7.3 Typical outputs	6
8 Output Naming and Generated Files	6
9 Recommended Validation Flows	6
9.1 Test A: Environment import test	7
9.2 Test B: One-spectrum serial synthesis	7
9.3 Test C: Small parallel run	7
9.4 Test D: One-star line-by-line run	7
10 Troubleshooting	7
10.1 <code>ModuleNotFoundError</code>	7
10.2 Cannot parse MARCS model filename	7
10.3 No safe interpolation pair or No complete safe MARCS interpolation cube	7
10.4 Missing or empty spectrum output	7
10.5 Spectrum already exists followed by skip	8
11 Pre-run Checklist for Long Jobs	8
12 Best Practices	8
A Minimal <code>input.ts</code> Template	8
B Minimal <code>abu.ts</code> Template	8
C Operational Rule of Thumb	9

1 Purpose and Scope

This manual explains how to run the current `HMSpectralGun` workflows. The suite generates stellar synthetic spectra with `Turbospectrum` starting from atmospheric parameters, line lists, abundance patterns, and optional post-processing steps such as convolution, resampling, and noise injection.

The repository contains three main workflows:

Workflow	Role
<code>main.py</code>	Classical serial synthesis workflow. It reads <code>input.ts</code> and synthesizes spectra one after another.
<code>main_parallel.py</code>	Parallel synthesis workflow. It follows the same logic as <code>main.py</code> , but distributes independent jobs across multiple workers.
<code>line_chi2_on_the_fly.py</code>	Line-by-line abundance workflow. It builds trial grids in $[X/Fe]$, synthesizes spectra on the fly, and estimates the best fit using a chi-square criterion.

Important

`Turbospectrum` is an external dependency. `HMSpectralGun` prepares inputs, launch scripts, and post-processing products, but the physical synthesis is performed by the compiled `Turbospectrum` executables.

2 Recommended Quick Start

From the repository root, first validate the Python environment and then run a minimal single-spectrum test before launching a long batch.

```
python --version
python -c "import numpy, pandas, scipy, matplotlib; print('core stack: ok!)"
python main.py --input input.ts --progress
```

For line-by-line analysis, start from the example JSON configuration and restrict the first run to one target:

```
python line_chi2_on_the_fly.py --config line_fit_config.example.json --star <star_id>
```

Note

The first production rule is simple: never start a large batch before a one-spectrum or one-star validation run has produced a physically sensible output file and usable logs.

3 Requirements

3.1 Python environment

Python 3.10 or newer is recommended. The code uses the following Python packages:

- `numpy`, `pandas`, `tqdm`, and `mendeleev`;
- `scipy`, for resampling and optimization utilities;
- `matplotlib`, for line-by-line PDF summary reports;

- PyAstronomy, for instrumental convolution.

A minimal environment check is:

```
which python
python - <<'PY'
import numpy, pandas, scipy, matplotlib
print('core scientific stack: ok')
PY
```

If instrumental convolution is required, also check:

```
python - <<'PY'
import PyAstronomy
print('PyAstronomy: ok')
PY
```

3.2 Turbospectrum installation

Turbospectrum must be compiled and its executables must be reachable through the paths expected by the code.

Important

Several paths are hardcoded in the current codebase, including paths such as `launchpath`, `dataset_model_path`, and the paths to `babsma_1u` and `bsyn_1u`. If the repository is moved to a different machine or directory, update these paths before running the workflows.

4 Input File Structure

4.1 The `input.ts` file

The file `input.ts` is used by `main.py` and `main_parallel.py`. The parser expects a rigid structure:

1. line 1: `savepath;`
2. line 2: `linelistpath;`
3. line 3: `modelpath;`
4. line 4: `ExplicitModel=True|False;`
5. line 5: `interp=True|False|nearest;`
6. line 6: third keyword, named NLTE in the code;
7. line 7 onward: spectrum table with either 15 or 17 columns.

Note

According to the available operational notes, the NLTE keyword is read by the parser but is not applied inside `TurboSpecWriter`. Treat it as non-operational unless the active repository version proves otherwise.

Minimal template:

```
/path/output_spectra/
/path/linelists/
/path/models/
ExplicitModel=False
interp=True
NLTE=False
```

```
3600,1.0 -1.00 0.20 15000 15500 2.0 st * 28000 0.02 * linelistH.ts abu.ts
* txt
```

4.2 Spectrum table columns

The spectrum table starts after the first six lines of `input.ts`. The expected column order is:

No.	Column	Meaning
1	Model	Explicit model filename, or <code>Teff,logg</code> .
2	[Fe/H]	Model metallicity.
3	[a/Fe]	Alpha enhancement.
4	lam_i	Initial wavelength.
5	lam_f	Final wavelength.
6	xi	Microturbulent velocity.
7	chemistry	Chemistry pattern: <code>st</code> , <code>ap</code> , <code>ae</code> , <code>mc</code> , or <code>hc</code> .
8	sampl	<code>*</code> or resampling step in Angstrom.
9	RES	Resolving power used for convolution.
10	resnum	Numerical wavelength step used in the Turbospectrum synthesis.
11	monoelem	<code>*</code> , atomic species such as <code>FeI</code> , or molecules such as <code>CO</code> , <code>OH</code> , <code>CN</code> .
12	linelist_file	File listing the line-list files to be used.
13	abu_file	Abundance file.
14	snr	<code>*</code> or numerical signal-to-noise ratio.
15	extension	Output extension, for example <code>txt</code> .
16	override_elem	Optional atomic number of the element to override.
17	override_xfe	Optional <code>[X/Fe]</code> value to force.

If columns 16 and 17 are provided, the override has maximum priority over the abundance file.

4.3 Abundance file: `abu.ts`

The abundance file is a two-column, whitespace-separated table:

- column 1: atomic number;
- column 2: abundance offset `[X/Fe]`.

Example:

```
8 0.20
12 0.20
26 0.00
612613 15.0
```

The special row `612613` is interpreted as the isotopic ratio `C12/C13`.

4.4 Line-list files

The `linelist_file` entry points to a plain-text file containing one line-list filename per line:

```
vald_1.lin  
vald_2.lin
```

In the classical synthesis workflows, these files are resolved relative to `linelistpath`. In the chi-square workflow, they are resolved relative to `linelist_library_path`.

5 Workflow 1: Classical Serial Synthesis

5.1 Command

Run the serial workflow with:

```
python main.py --input input.ts
```

To display a progress bar:

```
python main.py --input input.ts --progress
```

5.2 Execution logic

The serial workflow performs the following operations for each spectrum row:

1. selects or prepares the atmospheric model, using interpolation, nearest-neighbor selection, or an explicit model;
2. writes the Turbospectrum `.com` launch script;
3. runs Turbospectrum;
4. adds a descriptive header to the output spectrum;
5. optionally applies convolution, resampling, and noise injection.

Use this workflow when debugging a new configuration or when the number of spectra is small.

6 Workflow 2: Parallel Synthesis

6.1 Command

Run the parallel workflow with:

```
python main_parallel.py --input input.ts
```

6.2 Execution logic

The parallel workflow uses `ProcessPoolExecutor` with up to `cpu_count() - 1` workers. It is designed for independent spectra that can be synthesized concurrently.

Operational features include:

- worker-level logging;
- collision avoidance by renaming final outputs with a `_k<index>` suffix;
- a main log named `synth_main_YYYYMMDD_HHMMSS.log`.

Use this workflow for large batches when the input file has already been validated in serial mode.

7 Workflow 3: Line-by-line Chi-square Analysis

7.1 Configuration file

The line-by-line workflow is controlled by a JSON configuration file. A typical starting point is:

```
line_fit_config.example.json
```

Main configuration blocks include:

Block	Role
paths	Observed spectra, synthesis directory, and results directory.
keywords	Global options, including the [X/Fe] trial grid, fitting windows, interpolation mode, and related settings.
stars	List of targets to analyze.

7.2 Command

Basic execution:

```
python line_chi2_on_the_fly.py --config line_fit_config.example.json
```

Useful runtime overrides:

```
python line_chi2_on_the_fly.py --config cfg.json --star 301_4MCMC.H --xfe-min -0.5 --
  xfe-max 0.7 --xfe-step 0.1
```

7.3 Typical outputs

The usual outputs in `results_path` are:

- `<star>_best.csv`;
- `<star>_grid.csv`;
- `<star>_summary.pdf`, if `matplotlib` is available;
- legacy files named `results_<id>`.

8 Output Naming and Generated Files

Turbospectrum launch scripts are written as `<name>.com` files in `launchpath`. Final spectra are saved in `savepath` for the classical workflows, or under `analysis_synth_path/<star>/` for the chi-square workflow.

If `override_elem` and `override_xfe` are active, the generated filename includes an abundance tag such as `_Fep010` or `_Mgm025`.

Note

If a spectrum already exists and is non-empty, the workflow may skip its regeneration. This is intentional behavior and prevents accidental overwriting during repeated runs.

9 Recommended Validation Flows

Run tests in increasing order of cost.

9.1 Test A: Environment import test

```
python -c "import numpy, pandas, scipy, matplotlib; print('ok')"
```

9.2 Test B: One-spectrum serial synthesis

Prepare an `input.ts` file with a single spectrum row, then run:

```
python main.py --input input.ts --progress
```

Check that the output spectrum exists, is non-empty, and contains the expected header.

9.3 Test C: Small parallel run

After Test B passes, use a small multi-row input file:

```
python main_parallel.py --input input.ts
```

Check the main log and verify that output filenames do not collide.

9.4 Test D: One-star line-by-line run

Run one target with a narrow trial grid:

```
python line_chi2_on_the_fly.py --config cfg.json --star <star_id> --xfe-min -0.2 --xfe-max 0.2 --xfe-step 0.1
```

Inspect `<star>_best.csv`, `<star>_grid.csv`, and the summary PDF if generated.

10 Troubleshooting

10.1 ModuleNotFoundError

A Python dependency is missing from the active environment. Confirm the interpreter and install the missing package in that same environment.

```
which python
python -c "import numpy, pandas, scipy, matplotlib; print('ok')"
```

10.2 Cannot parse MARCS model filename

The atmospheric model filename does not match the expected naming format. Use a valid MARCS filename or switch to a supported explicit model configuration.

10.3 No safe interpolation pair or No complete safe MARCS interpolation cube

The MARCS model dataset is insufficient for the requested atmospheric parameters. Add the missing models, change the requested parameters, or use nearest-neighbor selection if scientifically acceptable for the test.

10.4 Missing or empty spectrum output

Turbospectrum likely failed or did not write the expected output file. Check the generated `.com` file and the Turbospectrum log `.txt` files in `launchpath`.

10.5 Spectrum already exists followed by skip

The output file already exists and is non-empty. This is expected skip behavior. Remove the existing output only if regeneration is intended.

11 Pre-run Checklist for Long Jobs

Before launching a long synthesis or line-by-line run, verify the following:

- absolute paths have been updated for the current machine;
- `contopac` is available or can be created;
- `input.ts` or the JSON configuration has been validated;
- line-list files resolve correctly;
- `abu.ts` resolves correctly and contains the intended abundances;
- Turbospectrum executables are reachable;
- a one-spectrum or one-star test has completed successfully;
- output directories are writable;
- old non-empty outputs have been removed if regeneration is required.

12 Best Practices

- Use `main.py` first when debugging a new input file; move to `main_parallel.py` only after the serial run works.
- Keep one minimal validation `input.ts` under version control for regression tests.
- Keep production outputs in dated directories to avoid mixing incompatible runs.
- Treat hardcoded absolute paths as configuration debt: document every required local edit before a run.
- For line-by-line analysis, start with a coarse $[X/Fe]$ grid and refine only after confirming that the observed and synthetic spectra are aligned.

A Minimal `input.ts` Template

```
/path/output_spectra/
/path/linelists/
/path/models/
ExplicitModel=False
interp=True
NLTE=False
3600,1.0 -1.00 0.20 15000 15500 2.0 st * 28000 0.02 * linelistH.ts abu.ts
* txt
```

B Minimal `abu.ts` Template

```
8 0.20
12 0.20
26 0.00
612613 15.0
```

C Operational Rule of Thumb

For development and debugging, prefer the safest sequence:

```
# 1. Validate imports
python -c "import numpy, pandas, scipy, matplotlib; print('ok')"
```

```
# 2. Run one serial spectrum
python main.py --input input.ts --progress
```

```
# 3. Only then run parallel or line-by-line workflows
python main_parallel.py --input input.ts
python line_chi2_on_the_fly.py --config cfg.json --star <star_id>
```